



Index Table

Section	Topic	Page Number
1. Overview	Definition	2
2. Prerequisites	Security Admin Role	2
3. Creation	How to Create	2
4. Access Levels	Table Level Access	4
	<i>Field Level Access</i>	6
	Global Level Access	8
5. Core Logic	Permission Flow	9
6. Operations	Operations	10
7. Rule Types	Scripted & Conditional ACLs	11

ACLs (Access Control List)

ACL (Access Control List) is a security rule that controls user access to data by defining who can perform specific operations like reading, writing, creating, or deleting records on tables and fields.

ACLs work by evaluating a set of conditions, roles, or scripts against a user's request for access.

For a user to perform an action (like viewing or editing a record), every ACL rule that applies must grant access.

Note → You need to have a security_admin role in order to create, update or delete an ACL.

The security_admin role in ServiceNow is an **elevated privilege**, meaning it provides higher-level security access.

Although many administrators have it, it is not automatically granted to every admin.

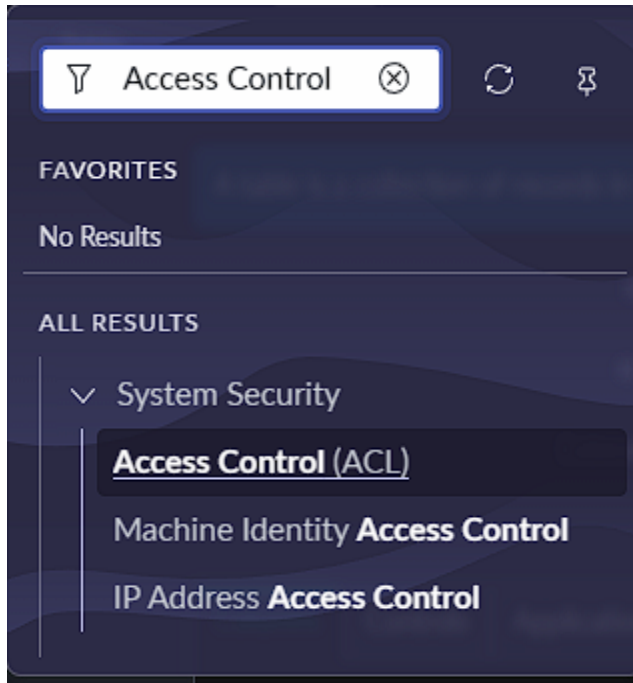
To work with sensitive security features—such as creating or modifying ACLs—you must:

1. Have the admin role, and
2. Temporarily elevate your privileges to security_admin using the “Elevate Roles” option.

This prevents accidental or unauthorized changes to critical security configurations.

There are 2 ways to create ACLs :-

1. From System Security → Access Control (ACL)



2. From the Table's Configuration in which you want to create Access Control.

[Explore REST API](#)
[\[SN Utils\] Collection Dictionary Entry](#)
[\[SN Utils\] Versions \(0\)](#)

Access Controls (103)	Security Data Filters	Labels (1)	Database Indexes (46)	Table Subscription Configuration (1)
-----------------------	-----------------------	------------	-----------------------	--------------------------------------

Name

Search

Name	Decision Type	Operation	Type	Active
incident	Deny Unless	report_view	record	true
incident	Allow If	report_view	record	true
incident	Deny Unless	write	record	true
incident	Allow If	read	record	true
incident	Allow If	write	record	true
incident	Allow If	read	record	true
incident	Allow If	write	record	true
incident	Allow If	write	record	true
incident	Allow If	delete	record	true
incident	Allow If	read	record	true
incident	Allow If	read	record	true
incident	Allow If	create	record	true

Level of Access

1. **Table Level Access** → Table-level access controls what actions a user can perform on an entire table—not just on a single record or field.

It can be applied using **TABLE.none**

* Type record

* Operation create

Decision Type Allow If

Admin overrides ☒

Protection policy -- None --

* Name Incident [incident] -- None --

Description

The core permissions of table level access are :-

- A. **Create** → Controls whether the user can add new records to the table.
It controls only the insertion of the records in a particular table.

For Example,

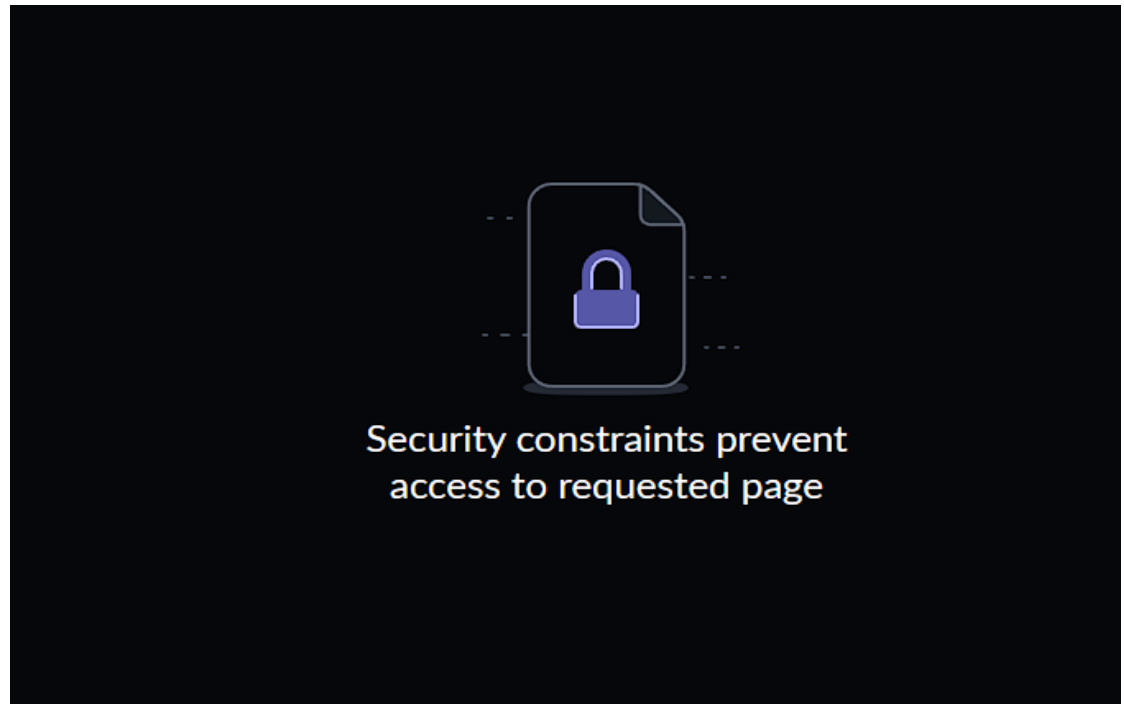
If the Create ACL on the incident table denies access, the user cannot open the “New” form.

Actions on selected rows...			
Date of Assignment	Role	Status	Assignment ID

- B. **Read** → Determines whether a user can view records in that table.

Example: If the user doesn't have Read access to the Incident, they will not see any Incident records—even if a list or report tries to show them.

You will get this type of error if you try to access the table you don't have read access to.



C. **Write** → Determines Who is allowed to modify existing records in a specific table.

Important Note →

Table-level update → permission to attempt the update

While,

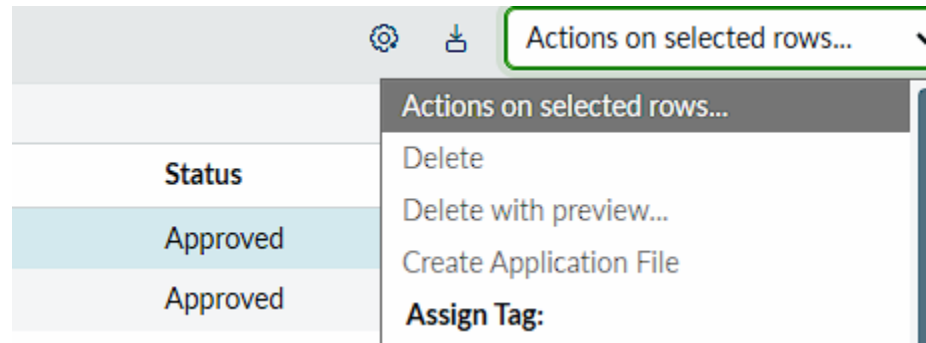
Field-level update → permission to edit specific fields

When you try to update a record, all the fields in it become readonly and there will be no update button when the update access is denied.

D. **Delete** → Determines Who is allowed to delete (permanently remove) a record from a specific table.

If the Delete ACL fails → the user cannot remove the record, even if they created it.

The Delete option gets Read only when the access is denied or from the record, the delete option will not be shown.



2. **Field Level Access** → A field-level ACL controls what a user can do on a specific field of a table.

It can be applied using

- Table.*

A screenshot of a form for configuring field-level access. The form has several fields: '* Type' with a dropdown menu set to 'record'; '* Operation' with a dropdown menu set to 'create'; 'Decision Type' with a dropdown menu set to 'Allow If'; 'Admin overrides' with a checked checkbox; 'Protection policy' with a dropdown menu set to '-- None --'; '* Name' with a dropdown menu set to 'Incident [incident]'; and a 'Description' field with a code icon and a text area. The '* Name' field is highlighted with a red rectangle.

- Table.{FieldName}

* Type ⓘ

* Operation ⓘ

Decision Type

Admin overrides ☒

Protection policy

* Name Actual end

Description 

The core permissions of field level access are :-

A. **Read** → It determines the visibility of the particular fields or the records.

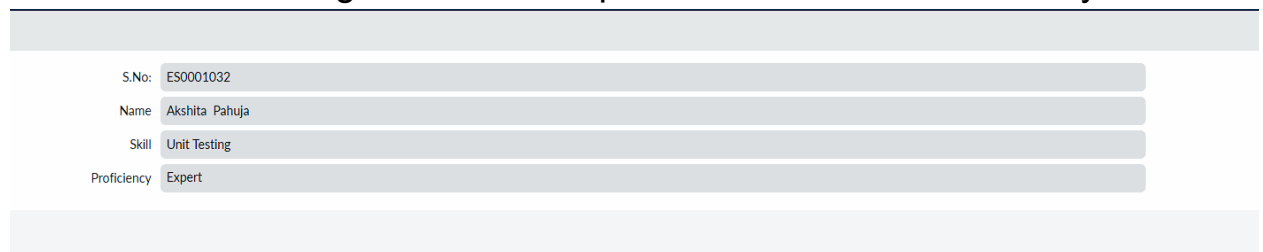
If a field Read ACL fails the field shows blank.



B. **Write** → Controls whether a user can modify a specific field on a record.

A user must pass **both** the table-level update ACL and the field-level update ACL to successfully edit a field.

If no field access is given then the specific fields become readonly.



C. **Create** → Decides whether a user is allowed to set a value in a specific field while creating a new record.

If the create access is denied at the field level then,

The field is not editable when creating a new record

The user sees:

- Field greyed out
- Field disabled
- Cannot type anything into the field

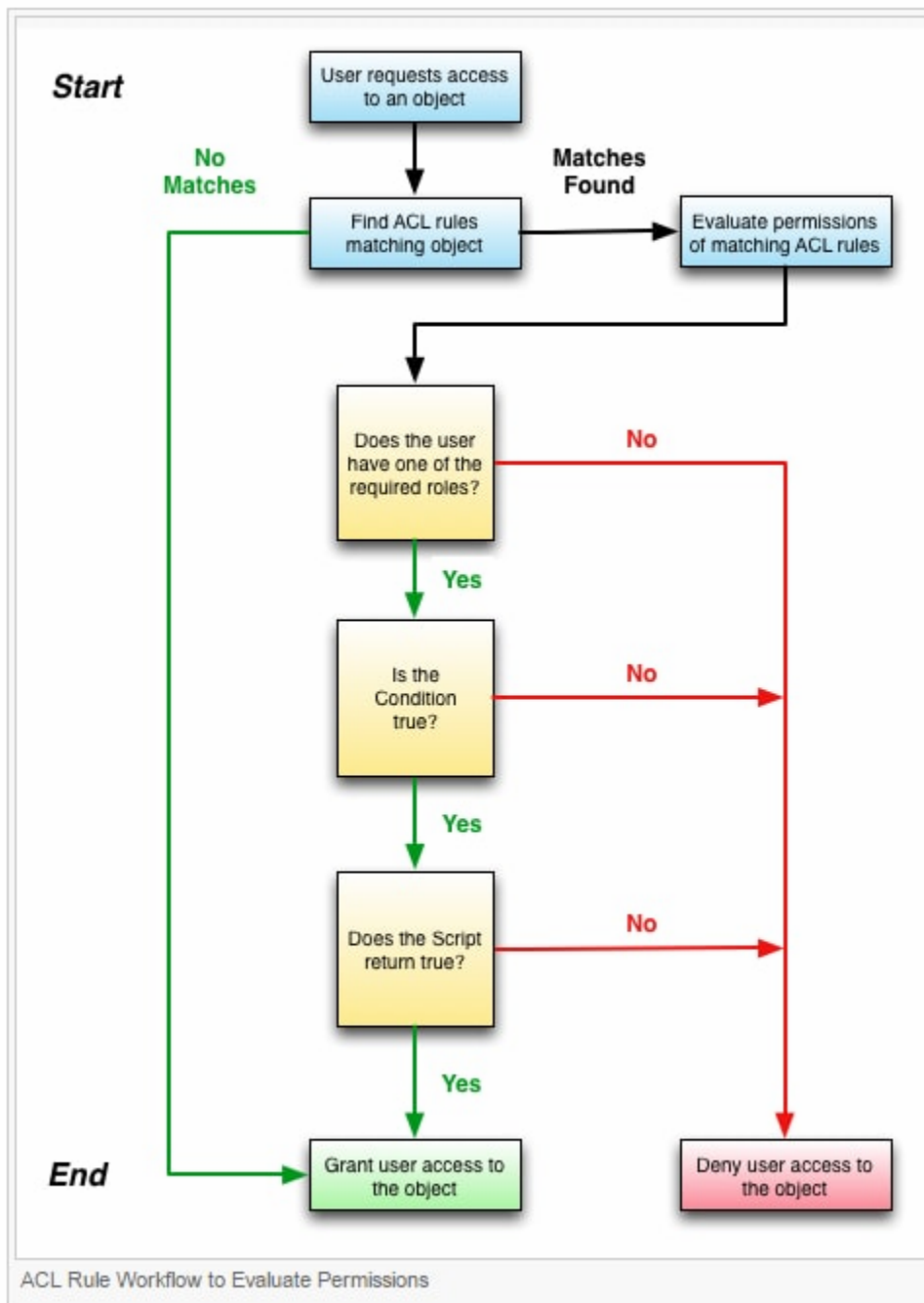
3.Global Level Access → Global Level Access is represented using (*.*) or (None.*)

. or (None.*) is a wildcard ACL that applies globally to all tables and all fields. It is the broadest ACL possible in ServiceNow.

Important Note → Read Permissions is the topmost permission a role should have in order to perform other operations.

If you do not have read access then, you will not be able to create, update, delete records whether you have access or not.

Flow of the ACLs



Operations of ACLs

Other than the CRUD operations, there are other operations as well which we can control using ACLs.

1. **list_edit (Inline Editing)** → Controls whether a user can edit fields directly from list view.

Example:

Editing short_description from the list (without opening the form).

Even if update ACL is allowed, list_edit is denied, will block inline editing.

2. **Execute** → Controls permission to execute server-side scripts.

Used for:

- Script Includes
- UI Actions (server-side)
- Business Rules

If the user fails to execute ACL, the script will not run.

3. **Save_as_template** → The save_as_template operation is a special ACL operation that controls whether a user can save a record as a template.

Templates are used to pre-fill forms with predefined field values, saving time for repetitive tasks.

ACLs Rule Type

Other than the Table Level and Field Level ACL, there are also other methods which can be used to set the ACLs rules.

1. **Role Based ACL** → It acts as the initial gatekeeper, ensuring the user has the necessary role to even be considered for access.

This is the first criteria evaluated by the system.

2. **Conditional ACL** → Uses a simple condition (like field value or boolean check) to allow/deny access.

Access Control Rules have two decision types, and these types will behave differently depending on conditions.

1. Allow Access: Allows access to a resource if all conditions are met. Additional ACLs may grant access to records where this ACL has not granted access.
2. Deny Access: Denies access to a resource unless all conditions are met. Additional ACLs may not grant access to records where this ACL has denied access.

[More Info](#)

Requires role

Role

+ Insert a new row...

Security Attribute Condition

Local or Existing ☒ Existing ☐ Local

Security Attribute

Condition All of these conditions must be met

-- choose field --

or

Data Condition

Condition [Not a valid table name](#)

Advanced Condition

Here, The Data Condition is used to control the ACL permissions based on certain conditions.

Advance Conditions is the scripting part.

Security Attributes are pre-defined or custom logical checks that validate the security context of a user session.

While a standard Condition checks the record (e.g., "Is the State Closed?"), a Security Attribute checks the user's session context (e.g., "Is this user coming from a trusted IP address?" or "Is this an interactive session?").

Its Conditions are evaluated at runtime → always up-to-date with current record values.

3. **Scripted ACL** → Uses a server-side script to dynamically determine access and can also combine roles and conditions.

Script returns true → access granted

Script returns false → access denied

Decision Types

There are two decision types on the basis of which the conditions are checked.

1. **Allow If** → Allow if grants access *only* if conditions are met.
It works in this way,
Allows access IF (roles match) AND (condition is true) AND (script returns true).
2. **Deny Unless** → Deny unless restrict the access unless a specific condition is matched.

Note → They sound the same, according to their definition but they have separate use cases.

Deny Unless is the first gatekeeper, blocking access by default unless a user passes specific criteria and Allow If then grants access based on its own conditions.

Deny Unless is evaluated first and acts as a filter (like a bouncer), whereas Allow If is a secondary filter (like a VIP list), granting access only if conditions pass, requiring both for comprehensive security.

They both work together.

ⓘ Warning: A role, security attribute, data condition, script or ACL control via reference fields is required to properly secure access with this ACL.

* Type	record	ⓘ
* Operation	create	ⓘ
Decision Type	Allow If	
Admin overrides	Allow If	
	Deny Unless	
Protection policy	None	